Архитектурный манифест: Декларативная федерация распределенных систем под управлением .NET Aspire

Введение

Данный документ представляет собой манифест проектирования современных распределенных систем высокой доступности и производительности в экосистеме .NET. Он агрегирует фундаментальные законы сетевого взаимодействия, описывает паттерны управления сквозной инфраструктурной сложностью и устанавливает баланс между синхронной композицией интерфейсов и асинхронной федерацией бизнес-логики.

В основе манифеста лежит концепция перехода от эмерджентной архитектуры (хаотично разрастающейся под влиянием локальных компромиссов) к декларативному дизайну среды, где топология приложения фиксируется как строгий исполняемый закон.

---

1. Код как конституция (Закон AppHost)

В современной среде .NET Aspire архитектура — это больше не просто абстрактная диаграмма в Visio; это исполняемый физический закон. Проект AppHost выступает в роли управляющего слоя (Control Plane), который жестко задает топологию сети.

Принципы изоляции сложности

- **Zero Trust по умолчанию. Сервисы физически не могут обнаруживать или вызывать друг друга, если им не предоставлено явное разрешение в AppHost через метод .WithReference(). Если связи нет в манифесте — её нет в реальности.**
- **Буфер гравитации (Борьба со спагетти). Центральный роутер/шлюз поглощает сетевую сложность, принудительно задавая топологию «Звезда» (Hub-and-Spoke). Это предотвращает появление хаотичной «сети смерти» (N × N соединений).**
- **Отсутствие скрытых путей. Без вызова .WithReference() адреса соседних сервисов и строки подключения просто не инжектируются в переменные окружения процесса. Попытка обойти правила и сделать прямой HTTP-вызов приведет к ошибке конфигурации еще на этапе локального запуска (dotnet run), а не на проме.**

csharp
// AppHost/Program.cs — Исполняемый манифест топологии системы

```
var builder = DistributedApplication.CreateBuilder(args);

// Ядро системы: Роутер имеет доступ ко всем базам данных и сервисам
var router = builder.AddProject<Projects.CentralRouter>("router");

// Сервисы состояний (Stateful services)
var ordersDb = builder.AddPostgres("ordersdb").AddDatabase("orders");
var balancesDb = builder.AddSqlServer("balancesdb").AddDatabase("money");

// Бизнес-сервисы: Полная изоляция друг от друга. Связь ТОЛЬКО с ядром.
var authService = builder.AddProject<Projects.AuthService>("auth")
                         .WithReference(router);

var tradeService = builder.AddProject<Projects.TradeService>("trade")
                          .WithReference(ordersDb)
                          .WithReference(balancesDb)
                          .WithReference(router);

// Фронтенд: знает только одну точку входа
var ui = builder.AddNpmApp("trading-ui", "../frontend")
                .WithHttpEndpoint(env: "VITE_API_URL", port: 80)
                .WithReference(router);

builder.Build().Run();
```

Используйте код с осторожностью.

---

2. Синхронная агрегация и асинхронные мутации

Центральный роутер работает как гибридный BFF (Backend-for-Frontend), разделяя пути чтения данных и изменения состояния:

🏛️ Путь чтения (Sync Path — API Composition)

Для построения UI-дашбордов без «проблемы N+1 запросов» роутер использует быстрый бинарный протокол gRPC (HTTP/2 мультиплексирование с Protobuf). Запросы разлетаются «веером» (fan-out) по внутренним сервисам параллельно через Task.WhenAll, агрегируя данные с минимальными накладными расходами на сериализацию.

📬 Путь записи (Async Path — Service Federation)

Для высоконагруженных мутаций (например, размещение ордера) используется связка WolverineFx и паттерна Transactional Outbox

- **Интерфейс мгновенно получает ответ HTTP 202 Accepted.**
- **Роутер транзакционно сохраняет сообщение в свою локальную БД.**
- **Фоновый воркер гарантирует асинхронную доставку во внутренние бизнес-сервисы, даже если они временно ушли в рестарт. Это обеспечивает 100% доступность системы на прием заявок.**

```
csharp
// Projects/CentralRouter/Program.cs — Реализация гибридного шлюза
var builder = WebApplication.CreateBuilder(args);

// gRPC-клиенты для внутренних синхронных вызовов
builder.Services.AddGrpcClient<Orderer.OrdererClient>(o =>
   o.Address = new Uri(builder.Configuration["services__trade__http"]));

// Настройка Wolverine для распределенной асинхронной шины
builder.Host.UseWolverine(opts =>
{

opts.PersistMessagesWithPostgres(builder.Configuration.GetConnectionString("or
ders"));
   opts.PublishMessage<OrderPlaced>().ToServiceName("trade");
});

var app = builder.Build();

// --- SYNC PATH: API Composition для UI ---
app.MapGet("/api/user-dashboard/{id}", async (Guid id, HttpContext ctx) =>
{
   var userTask = authClient.GetUserAsync(new GetUserRequest { Id = id });
   var portfolioTask = tradeClient.GetPortfolioAsync(new PortfolioRequest { Id = id
});

   await Task.WhenAll(userTask, portfolioTask);
   return Results.Ok(new { User = userTask.Result.User, Assets =
portfolioTask.Result.Assets });
});

// --- ASYNC PATH: Локальный Outbox для Highload-мутаций ---
app.MapPost("/api/place-order", (PlaceOrderCommand cmd, IMessageBus bus) =>
{
   bus.PublishAsync(cmd); // Ушло в локальную таблицу outbox_messages
```

```
    return Results.Accepted(); // Мгновенный ответ HTTP 202
})
.Produces(StatusCodes.Status202Accepted);

app.Run();
```

Используйте код с осторожностью.

---

3. Сверхнизколатентные структуры данных и Мэтчинг

Принципы изоляции сложности спускаются с уровня веб-коммуникаций на уровень самого производительного ядра системы — Matching Engine (движка сведения ордеров).

Чтобы избежать накладных расходов на блокировки (locks) и состояний гонки (race conditions), промышленный матчер работает строго в одном потоке, который жестко привязывается к изолированному ядру процессора (CPU affinity).

Организация памяти стакана (Order Book)

1. Уровни цен (Красно-черное дерево): Требуют быстрых вставок, удалений и постоянного доступа к лучшим ценам спроса/предложения (***best bid/ask***). Хранятся в самобалансирующемся бинарном дереве поиска (Price -> Queue), гарантируя сложность $O(\log N)$ для поиска новых уровней и O(1) для получения вершины стакана.
2. Очереди ордеров (Двусвязный список): Внутри конкретного ценового уровня ордера упорядочены в двусвязный список. Это обеспечивает сложность O(1) для добавления новых заявок в конец и O(1) для удаления (при отмене или полном исполнении, используя хэш-таблицу для прямой ссылки на узел списка).

```
[Красно-черное дерево цен]
        /     \
   Цена: $51  Цена: $50
                |
     [Двусвязный список ордеров]
     [Ордер A: 10] <=> [Ордер B: 5]
```

Алгоритмы сведения ордеров

- FIFO (Price-Time Priority): Стандарт для спотовых рынков. Ордера на одном ценевом уровне исполняются строго в порядке их времени

прихода. Метод поощряет маркетмейкеров, предоставляющих ликвидность раньше других.

- Pro-Rata (Пропорциональный метод): Популярен на рынках фьючерсов и опционов. Временной приоритет игнорируется: входящий объем распределяется пропорционально объему уже существующих лимитных заявок. Метод поощряет выставление глубокой, крупной ликвидности.

Разделение потоков Ingress / Egress

Матчер использует тот же паттерн асинхронного буфера, что и верхнеуровневый Роутер

- Входной трафик (Ingress): Заявки со шлюзов собираются низколатентным секвенсором (например, *LMAX Disruptor* или *Aeron*) для выстраивания строгого детерминированного порядка перед подачей в поток мэтчинга.
- Выходной трафик (Egress): Результаты сделок и обновления стакана выбрасываются из потока мэтчинга асинхронно через *UDP Multicast* или *Kafka* дистрибьюторам рыночных данных. Основной расчетный цикл никогда не блокируется сетевым вводом-выводом.

---

4. Семантика сетевых протоколов: Триумф URI и цена бинарных абстракций

4.1 URI как идеальный ключ маршрутизации

В распределенных архитектурных схемах (ServiceA <--> Router <--> ServiceB) центральному узлу каждый раз необходимо принимать решение: *«Куда именно отправить этот пакет?»*.

HTTP выиграл глобальную инфраструктурную войну не потому, что он «быстрый» (напротив, он медленный и тяжелый), а потому что его модель адресации на прикладном уровне (L7) совпадает с моделью человеческого мышления о ресурсах.

HTTP предложил миру гениальное разделение ответственности на сетевом уровне:

- Конверт (Headers + URI): Текстовый, прозрачный, легко читаемый любым промежуточным узлом.
- Письмо (Body): Любая произвольная полезная нагрузка (Payload).

Семантическая прозрачность

http

POST https://router.internal HTTP/1.1
Используйте код с осторожностью.
При таком подходе роутеру даже не требуется открывать и анализировать тело сообщения. Глядя исключительно на заголовок Host и строку запроса (*Request Target*), он мгновенно извлекает все необходимые компоненты:

- **Схема: https**
- **Хост (кластер): router.internal**
- **Путь (целевой ресурс): /api/orders/create**

Для принятия решения по маршрутизации через YARP или стандартное middleware достаточно распарсить строку длиной в несколько десятков байт. Это происходит на сетевом уровне до того, как сработает тяжелый десериализатор JSON или Protobuf. Роутер остается абсолютно абстрагированным («тупым») в отношении бизнес-логики, обеспечивая строгое разделение ответственности (*Separation of Concerns*).

4.2 Проблема «голого потока байтов» (Raw TCP)

Если компоненты системы взаимодействуют по чистому TCP-сокету, сеть видит лишь бесконечную трубу с единицами и нулями (голый поток байтов). Инфраструктурный инженер на уровне L4 видит только IP-адреса и порты, но полностью лишен контекста.

Чтобы стандартный балансировщик (например, Nginx или HAProxy) мог переслать этот пакет, разработчикам приходится изобретать собственные велосипеды: парсить первые 4 байта как длину сообщения, следующие 16 байт интерпретировать как «Ключ маршрутизации» (*Routing Key*), и только затем читать полезную нагрузку. Поскольку единого стандарта нет и каждый пишет свой парсер, маршрутизировать такой трафик на уровне L7 из коробки становится невозможно. Поддержка кастомных роутеров для бинарных потоков — это эксплуатационный ад.

4.3 Суррогаты URI (gRPC и RabbitMQ)

Вся индустрия бэкенд-разработки, пытаясь убежать от ограничений HTTP ради скорости или асинхронности, в итоге неизбежно воссоздавала концепцию HTTP заново, но в более сложных и менее стандартизированных формах.

Закон неизбежности иерархического адреса (Закон URI)

Любая система связи, которая масштабируется до уровня нескольких независимых узлов, неизбежно вынуждена внедрить механизм явной адресации ресурсов на прикладном уровне (L7). Если протокол этого не предоставляет изначально, разработчики построят его суррогат поверх существующего payload'а.

- **gRPC (Эмуляция путей): Google создал gRPC поверх HTTP/2, совершив сильный инженерный и политический компромисс. Метод OrderService.CreateOrder упаковывается в пакет HTTP/2, где в стандартный заголовок :path принудительно передается строка /v1.OrderService/CreateOrder. Маршрутизация gRPC всё равно опирается на виртуальный «путь», эмулирующий REST-иерархию ресурсов поверх RPC. Если бы этого пути не было, ID сервиса пришлось бы передавать в самом теле Protobuf-сообщения, смешивая транспорт и бизнес-логику. Маска HTTP/2 позволила gRPC проехать «зайцем» через жесткую мировую инфраструктуру прокси-серверов и фаерволов (*L4 firewalls*), которые иначе заблокировали бы неизвестные кастомные порты.**
- **WolverineFx / RabbitMQ (Маршрутизация очередей): В мире асинхронных очередей сообщение представляет собой безликий массив байтов. Чтобы оно попало в правильный сектор, инфраструктуре необходим заранее известный *Exchange* или *Routing Key* (работающий через Smart Broker). Этот Routing Key (например, строки вида order.created.v1) — не что иное, как воссозданный суррогат URI для мира очередей.**

4.4 Цена абстракции: Слепота инфраструктуры

На кастомных бинарных сокетах часто «горят» при масштабировании, платя за это высокой когнитивной нагрузкой и сетевой слепотой.

- **Автоматизация кэширования: Если вы используете HTTP GET /api/products/42, вам не нужно писать код для кэширования. Вы декларативно выставляете перед сервисом Redis, Varnish или Cloudflare, и они кэшируют данные автоматически по факту совпадения URI-строки. В кастомном бинарном протоколе инфраструктура слепа: чтобы закэшировать «Вызов метода №42», логику кэширования и инвалидации приходится писать руками внутри бизнес-кода самого приложения.**
- **Управление трафиком: Прозрачность L7 позволяет балансировщикам сбрасывать тяжелые запросы к дебаг-эндпоинтам (/api/debug) при перегрузках, сохраняя работоспособность легких критических путей (/api/pay). В бинарном сокете для этого пришлось бы писать сложный stateful-файервол, досконально понимающий внутренмый формат байтов.**

4.5 Роль исторических компромиссов и .NET Aspire

Использование HTTP/2 в качестве подложки позволило gRPC выжить на рынке и пройти «зайцем» через жесткую мировую инфраструктуру прокси-серверов и корпоративных фаерволов. Если бы Google попытался навязать миру кастомный L4-транспорт и свои порты, админы просто заблокировали бы неизвестный бинарный шум.

Именно поэтому оркестратор .NET Aspire делает ставку на именованные конечные точки (*endpoints*) и стандартные примитивы HttpClient:

csharp
var payment = builder.AddProject<Projects.Payment>("pay");
var orders =
builder.AddProject<Projects.Orders>("orders").WithReference(payment);
Используйте код с осторожностью.
Среда автоматически подставляет адрес https://payment:7262 в переменные окружения зависимого сервиса, поскольку спецификация того, что такое «адрес», давно стандартизирована в RFC. Мы платим осознанный налог на производительность (затраты процессора на парсинг URI-строк и текстовых заголовков) ради колоссальной выгоды в управляемости, автоматическом Service Discovery, отладке и безопасности распределенной среды. Это триумф прагматизма над перфекционизмом.

---

5. Эволюция внутреннего транспорта в .NET Aspire: Борьба с избыточным проектированием

5.1 Проблема овер-инжиниринга распределенных систем

То, что экосистема .NET Aspire позволяет одной строчкой кода (например, builder.AddRabbitMQ()) мгновенно поднять полноценную инфраструктуру, сыграло с индустрией злую шутку. Разработчики начали внедрять тяжеловесные брокеры сообщений (RabbitMQ, Kafka) и абстракции (MassTransit) повсеместно — даже там, где два соседних компонента просто хотят обменяться базовыми уведомлениями.

Внедрение внешних брокеров для коммуникации элементов, которые живут внутри единой оркестрируемой среды разработки, — это классический Over-engineering (избыточное проектирование). Использование таких инструментов оправдано для обмена данными за пределами внешних границ .NET Aspire, но внутреннее взаимодействие должно быть организовано на базе более производительных и легковесных компонентов.

5.2 Границы процессов и сетевой налог

Главный нюанс локальной оркестрации в .NET Aspire заключается в границах процессов (Process Boundaries). Даже внутри единого хоста каждый

проект запускается как изолированный процесс (разные контейнеры или исполняемые файлы .exe). Они не делят общую оперативную память.

По этой причине невозможно использовать стандартный локальный EventBus (например, public static class или чистый MediatR в памяти) для межпроектной связи — компонентам в любом случае необходим сетевой транспорт, чтобы передать байты от процесса к процессу. Однако этот транспорт можно сделать максимально эффективным.

5.3 Альтернативные стратегии взаимодействия «Сервер-Сервер»

Если отказаться от громоздкой инфраструктуры в пользу прагматичных решений, для внутреннего общения в .NET Aspire остаются три оптимальных пути:

⚡ **gRPC (Абсолютный чемпион для Point-to-Point)**

Когда сервису (например, API Gateway) нужно передать четкую команду конкретному исполнителю (например, Matching Engine), распределенная очередь не нужна. Необходим быстрый бинарный вызов.

- **Service Discovery: Экосистема .NET Aspire предоставляет нативную интеграцию — достаточно объявить .WithReference(), и gRPC-клиент автоматически найдет нужный порт без жестко зашитых конфигураций.**
- **Производительность: Работа поверх HTTP/2 с сериализацией в бинарный Protobuf обеспечивает максимальную скорость и минимальный налог на процессорное время.**
- **Наблюдаемость: gRPC-вызовы из коробки идеально визуализируются на .NET Aspire Dashboard в разделе распределенной трассировки (*Distributed Tracing*).**

🚀 **NATS (Легковесный Pub/Sub для распределенных событий)**

Если в системе необходим чистый паттерн «Издатель-Подписчик» (например, когда движок мэтчинга исполнил сделку, и об этом должны одновременно узнать сервисы Балансов, Истории и Уведомлений), gRPC не подойдет, так как он работает строго в режиме «один-к-одному».

- **В качестве альтернативы RabbitMQ выбирается NATS — крошечный брокер сообщений, написанный на Go.**
- **Он обеспечивает прокачку миллионов сообщений в секунду за счет отсутствия сложной enterprise-логики (роутингов, сложных обменников и персистентных стратегий). Это сверхбыстрая и чистая «труба» для доставки событий.**

🏛️ **Модульный монолит (In-Memory Channels)**

Архитектурный подход, полностью ликвидирующий сетевой налог. Система проектируется как единый запускаемый проект-хост в .NET Aspire, но внутри логика разделяется на изолированные модули (разные сборки .dll).

- **В такой конфигурации EventBus в памяти работает со скоростью света.**
- **Взаимодействие строится на базе System.Threading.Channels (встроенные высокопроизводительные очереди в .NET) или локального MediatR. Вся коммуникация занимает наносекунды, так как полностью отсутствуют сетевые скачки и сериализация данных в JSON/Protobuf.**

5.4 Специфика SignalR в архитектуре хоста

Технологию SignalR необходимо четко разделять по зонам ответственности:

1. **Взаимодействие «Сервер -> Тонкий Клиент» (Идеально): Для связи бэкенда с внешним интерфейсом (React/Blazor SPA) SignalR является абсолютным лидером. Через него торговый сервер эффективно транслирует (*пушит*) обновления стакана ордеров, тики цен и статусы транзакций в браузер пользователя, инкапсулируя управление WebSockets и восстановление соединений.**
2. **Взаимодействие «Сервер -> Сервер» (С ограничениями): Использовать SignalR Hub для связи бэкенд-процессов внутри .NET Aspire технически возможно, но не рекомендуется. Настройка долгоживущих WebSocket-соединений между соседними контейнерами создает избыточный оверхед (ресурсы тратятся на хэндшейки, пинги и heartbeats). Кроме того, чистый SignalR работает по принципу *At-most-once* (доставка максимум один раз) и не гарантирует сохранность сообщений: если принимающий worker ушел в рестарт, отправленное в этот момент событие будет потеряно навсегда.**

5.5 Идеальная абстракция: WolverineFx

Библиотека WolverineFx решает фундаментальную задачу — она выступает высокоуровневой абстракцией над инфраструктурой транспорта, избавляя проект от сложного «церемониального» кода (чем часто страдает MassTransit).

Ключевые преимущества WolverineFx

- **Прозрачность маршрутизации: Бизнес-код оперирует простой командой bus.Publish(new OrderPlacedEvent()). WolverineFx на этапе компиляции с помощью Source Generators генерирует оптимальный прокси-код: если подписчик живет в том же процессе, вызов превращается в in-memory обработку через Channels, если в соседнем — упаковывается в сеть.**
- **Нативная интеграция: Поддержка стандартов OpenTelemetry гарантирует, что каждое сообщение (даже прошедшее через локальную шину в памяти) будет отрисовано в виде сквозного трейса на .NET Aspire**

Dashboard. Вы визуально видите цепочку причинно-следственных связей (например, как вызов шлюза породил транзакцию в базе данных).

- **Синергия с экосистемой: Идеально сочетается с фреймворком Marten для построения высоконагруженных систем на базе *Event Sourcing` и CQRS, обеспечивая работу транзакционного паттерна Outbox прямо «из коробки».**

---

6. Триада управления распределенной системой

Итоговая концепция Декларативной федерации полностью закрывает инфраструктурные потребности enterprise-систем за счет трех опорных точек:

Компонент	Инструмент .NET	Роль в архитектуре
РАДАР	**OpenTelemetry + Aspire Dashboard**	**Полная наблюдаемость. Сквозной Distributed Tracing проходит через «стеклянный бокс» шлюза, подсвечивая точные задержки каждого внутреннего gRPC-скачка или сбоя.**
РУЛЬ	**AppHost (IDistributedApplicationBuilder)**	**Управление топологией. Конфигурация сети является исполняемым C#-кодом. Изменение направления потока данных или изоляция узла делаются в одной точке.**
БРОНЯ	**Central Router + Wolverine Outbox**	**Отказоустойчивость. Падение любого вторичного микросервиса (например, истории или аналитики) полностью амортизируется Outbox-хранилищем ядра, сохраняя систему доступной для пользователей.**

Заключение: Архитектурная преемственность от AppHost к CPU Core

Разработанный паттерн «Декларативная федерация» демонстрирует сквозную фрактальную преемственность: одна и та же концепция изоляции сложности и бесконфликтного упорядочивания повторяется на всех уровнях работающей системы.

- **На уровне Control Plane (AppHost): Мы используем WithReference(), чтобы заблокировать хаос связей N × N и гарантировать Zero Trust на этапе сборки.**
- **На уровне Data Plane (Central Router): Мы изолируем межсервисные скачки за прозрачным L7-маршрутизатором и буферизируем исходящие транзакции мутаций через Wolverine Outbox.**
- **На уровне High-Load ядра (Matching Engine): Тот же самый паттерн повторяется внутри одного процесса! Поток мэтчинга изолируется на выделенном ядре процессора (CPU Affinity), а входящий трафик выстраивается в строгую детерминированную очередь через кольцевой буфер LMAX Disruptor (Ingress). Результаты обработки сбрасываются в асинхронный выходной канал (Egress), никогда не блокируя основной расчетный цикл.**

Мы платим осознанный налог на абстракцию в виде парсинга URI-строк прикладного уровня, но получаем абсолютную управляемость, детерминированную безопасность и сквозной мониторинг, превращающие распределенную систему в предсказуемый и надежный инженерный комплекс, способный пережить смену поколений кодовой базы.

Итоговый объединенный Архитектурный манифест декларативной федерации полностью собран из всех четырех правильных файлов без пропусков и готов к копированию.